CS683 PA-1

Task 1 Report

2D Convolution

24B1021 | 24B1015 | 24B1037 | 24B0995

Loop Reordering · Loop Unrolling · Cache Tiling · SIMD (AVX2)

September 7, 2026

Contents

Setup

All runs were on an Intel 13th Gen CPU, which has both performance (P) cores and efficiency (E) cores. The P-cores have a 48 KB L1-D cache, and the E-cores have a 32 KB L1-D cache. All the measurements in this report were run on CPU 0, which is a P-core.

1 Task 1A: Loop Reordering & Unrolling

1.1 Loop Reordering

In `conv_naive` the loop order is $oy \to ox \to ky \to kx$: for every output pixel we loop over all K^2 kernel entries. This means the inner loop jumps to a different input row for each ky, giving strided (non-sequential) reads and a scattered write pattern to the accumulator. The reordered version moves the kernel loops outside: $ky \to kx \to oy \to ox$. The kernel weight `ker[ky*K+kx]` is calculated outside as a scalar, and the innermost loop over ox becomes a unit-stride sequential read of a row of the input and a sequential read-modify-write of the output row, which is better for the hardware prefetcher.

Considerations

The reorder requires zeroing the output first (since we accumulate across K^2 passes rather than computing each pixel completely before moving on). The downside is that the output image is touched K^2 times across those passes; at large image sizes this causes repeated cache evictions of the output array.

Results

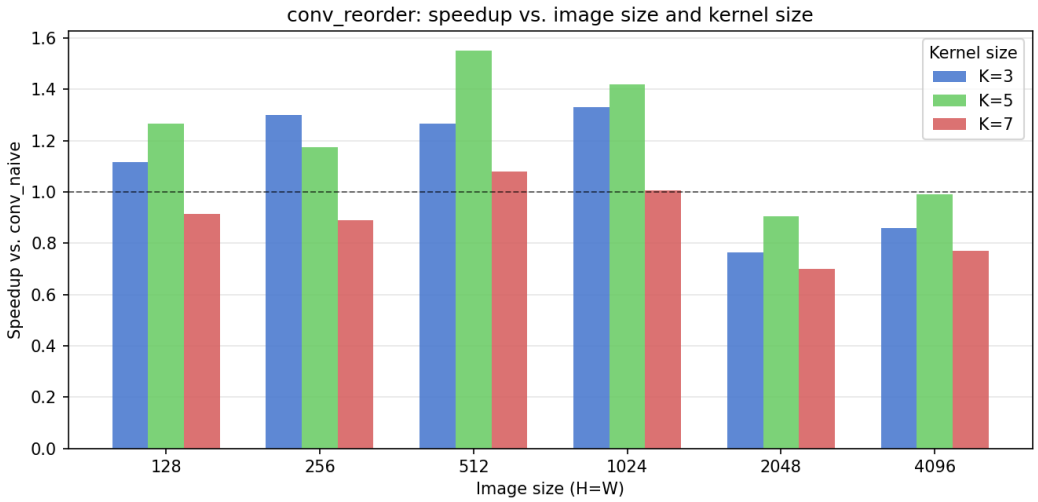

Figure 1: Speedup of `conv_reorder` over `conv_naive` vs. image size, grouped by kernel size K.

We use speedup over `conv_naive` as the metric since it directly captures runtime benefit relative to the baseline.

At $K = 3$, `conv_reorder` is *slower* than naive for large images ($0.75\times$ at 2048^2, $0.85\times$ at 4096^2) but faster for smaller ones ($1.3\times$ at 1024^2). The reason is the $K^2 = 9$ passes over the output: for a 2048×2048 image the output is 16 MB, which does not fit in L2 or L3, so each pass causes cache misses when re-reading the output to accumulate. The naive loop touches

each output element exactly once (completing it before moving on), so it has far fewer output reads. At smaller image sizes the output fits in cache, the K^2 passes stay resident, and the unit-stride access pattern of the reorder pays off.

1.2 Loop Unrolling

`conv_unroll` keeps the same $oy \to ox \to ky \to kx$ loop order as `conv_naive` but unrolls the ox loop by a factor of 4, processing four output pixels per outer iteration. Each of the four pixels gets its own accumulator (`acc0`–`acc3`) that accumulates independently over the K^2 kernel values. Because these accumulators are independent of each other, the CPU can do multiple add operations simultaneously, hiding the latency.

Results

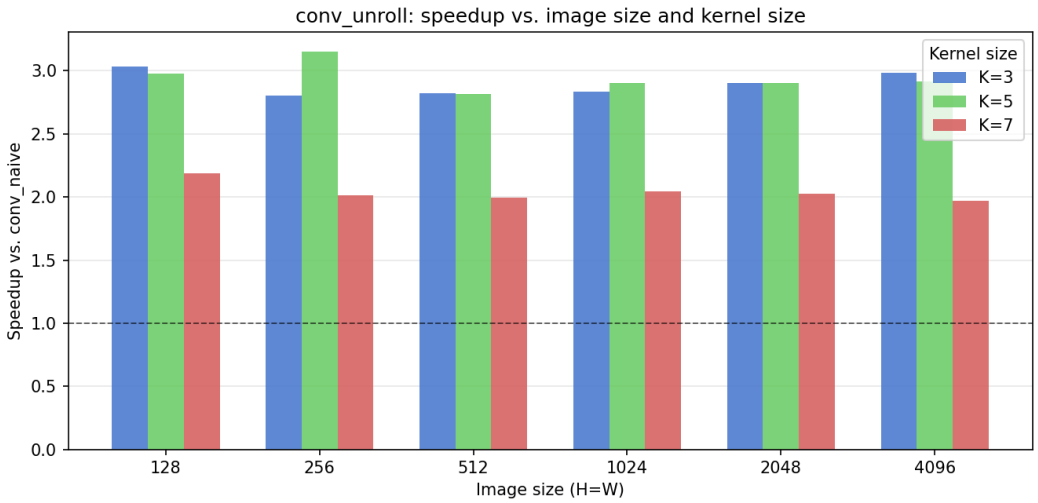

Figure 2: Speedup of `conv_unroll` over `conv_naive` vs. image size, grouped by kernel size K.

Discussion

Because the output is written exactly once (like naive), there is no multi-pass cache eviction problem, and the speedup is positive across all tested sizes. The gain comes entirely from the CPU's instruction level parallelism: four independent accumulators let the CPU overlap additions instead of executing them in a latency-bound chain. The speedup is relatively consistent across image sizes since the memory access pattern is identical to naive and both are equally prefetcher-friendly.

2 Task 1B: Tiling

2.1 Baseline Profiling

The CPU we used to run is the same as Task 1A. Again, all were run on core 0 (P-core).

```
taskset -c 0 perf stat -r 5 \
    -e instructions,L1-dcache-loads,L1-dcache-load-misses \
    ./bin/conv naive
```

Metric	Value (5-run mean)
Instructions	4,371,265,012
L1d loads	892,590,815
L1d load misses	571,072
Time elapsed (s)	0.222255 ± 0.002410

Table 1: `perf stat` results for `conv_naive` on a 2048×2048, $K = 3$ workload, pinned to P-core CPU 0.

$$\text{MPKI} = \frac{\text{L1-D load misses}}{\text{instructions}} \times 1000 = \frac{571{,}072}{4{,}371{,}265{,}012} \times 1000 \approx 0.131$$

Key Insight

An MPKI of ≈ 0.13 is pretty low, and it's consistent with what the naive kernel is actually doing. For every output pixel, it just needs a tiny 3×3 neighborhood of the input and the same 3×3 kernel it already used one pixel ago. All of that easily fits into the cache. So even though we loop through the entire 16 MB image over the course of the run, at any given moment it's only really touching a small, reused patch of memory. Hence, we don't get as many misses in this case. The naive implementation, since it has sequential accesses, is also pretty hardware prefetcher friendly so that also contributes to a low MPKI.

2.2 Tiled Implementation

`conv_tile` blocks the output into square $T \times T$ tiles, running all $K \times K$ taps for a tile before moving on, so the tile stays resident in L1-D instead of being re-swept from DRAM $K \times K$ times like `conv_reorder`. We swept T on the workload (2048×2048, $K = 3$):

T	Time (ms)	Speedup	L1-D MPKI
8	18.580	0.92×	0.920
16	17.451	0.93×	21.533
24	15.642	1.03×	24.495
32	16.225	1.05×	19.624
48	15.073	1.16×	16.421
64	**14.809**	**1.10×**	**14.148**
80	15.479	1.06×	9.852

Table 2: `conv_tile` tile-size sweep, pinned to P-core CPU 0.

$T = 64$ is fastest and fits the 48 KB L1-D (≈ 33.4 KB working set).

Key Insight

Fastest ($T = 64$) isn't the same as fewest misses ($T = 8$, MPKI ≈ 0.92): tiny tiles avoid conflict misses but pay for it in loop overhead, T from 16 to 48 all have high MPKI but $T = 64$ is the best tradeoff between the two. $T = 80$ has an even lower MPKI, but its

≈ 52 KB working set no longer fits the 48 KB L1-D, so the capacity misses it introduces cost more in real latency than the MPKI ratio alone shows, making it slower than $T = 64$ despite "looking" more cache-efficient.

2.3 Metrics and Plots

We swept image size $\in \{128, 256, 512, 1024, 2048, 4096\}$ ($K = 3$) for three tile sizes: $T = 8$ (best MPKI), $T = 24$ (worst, mid conflict-miss zone), and $T = 64$, against the naive baseline:

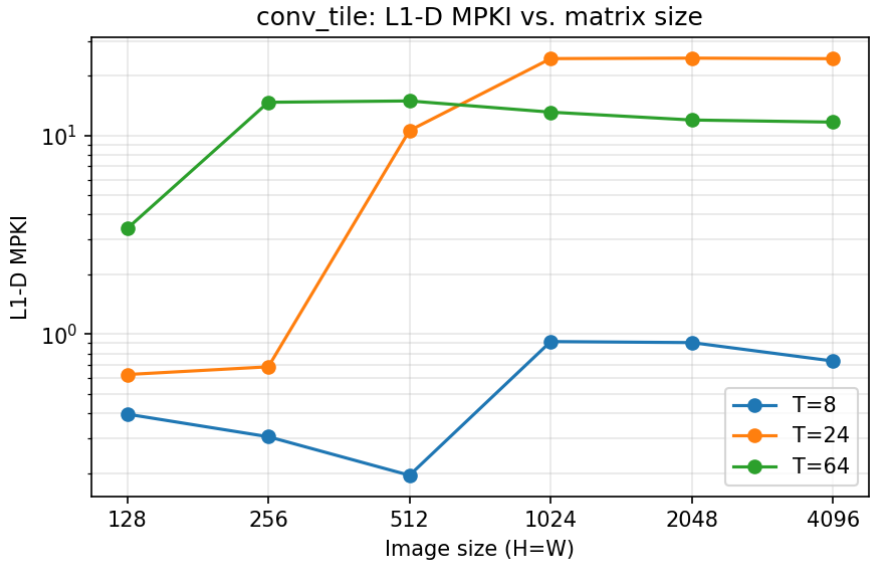

Figure 3: L1-D MPKI vs. matrix size, one line per tile size.

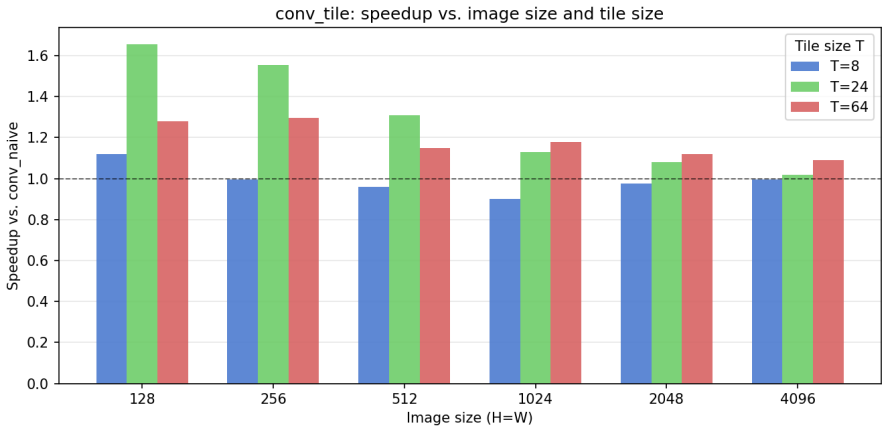

Figure 4: Speedup vs. matrix size, one line per tile size.

2.4 Questions

1. **MPKI change naive → tiled:** The MPKI, as we increase the tile size, gets worse than the naive implementation. This could be because the naive implementation is already pretty cache-optimal at $K = 3$. It benefits from long, contiguous horizontal memory accesses, which the prefetcher could easily predict and optimize. By using tiling, these contiguous accesses are broken into short burst accesses. Moreover, the working set for the naive approach

probably already fits well into the L1D cache.

2. **MPKI vs. matrix size/tile size:** For $T = 8$, L1-D MPKI remains consistently low across all matrix sizes. For $T = 24$ to $T = 64$, MPKI starts low but spikes drastically as the matrix grows and finally plateaus. This is because, at the smallest matrix size, the entire working set fits into L1-D cache so misses are rare. As the matrix grows, the data exceeds L1-D capacity. The $T = 8$ tile size keeps its active working set small enough to remain in cache. But larger tile sizes introduce vertical memory jumps that exceed cache capacity, causing misses and moreover, it isn't prefetcher friendly.

3. **Speedup achieved:** Speedup was achieved only for larger tile sizes (> 24) with the sweet spot being $T = 64$. The performance improved despite high L1-D misses, indicating that tiling successfully improved temporal locality which led to lower latency even if L1-D misses made it fetch data from LLC. This could be because we don't have to go to L3 or DRAM to fetch something we use in a "short" span of time because we reuse that data often (temporal locality). Also the number of instructions of tiling is about half that of the naive implementation so that is also a factor which contributes to the speedup. One of the ways to make this even more efficient could be loop unrolling (to abuse spacial locality along with temporal).

3 Task 1C: SIMD

3.1 Baseline Profiling

Since we're running the naive/simd function from the main function, there will be some extra instructions counted by `perf stat` for allocation, filling of arrays etc. So, we isolated the per-kernel instruction count with `perf record` on the 2048×2048, $K = 3$ workload, pinned to P-core CPU 0:

```
taskset -c 0 perf record -e instructions:u -o pr.naive ./bin/conv naive 2048 2048 3
perf report -i pr.naive --stdio --sort symbol --percent-limit 0.1
```

(and likewise with `./bin/conv simd 2048 2048 3` for the SIMD kernel.) In single-stage mode the main code invokes each convolution function a fixed number of times per run: **11** for `conv_naive` (one reference call in `run_config`, plus one untimed correctness call and `kWarmup`+`kReps`$= 2 + 7$ timed calls in the stage loop) and **10** for the stage under test. Dividing the per-symbol totals by these counts gives the per-call figures:

Code	Total instr.	Calls	Per call
`conv_naive`	3,910,423,333	11	355,493,030
`conv_simd` (256-bit)	435,688,491	10	43,568,849

Table 3: No. of instructions for naive and SIMD optimizations

3.2 Instruction Count Analysis

The 256-bit SIMD code executes ≈ 43.6 M instructions per call against the naive code's ≈ 355.5 M (≈ 8.2× reduction), essentially the 8-wide vector width.

3.3 Performance

Stage	Time (ms)	GFLOP/s	Speedup
`conv_naive`	14.779	5.11	1.00×
`conv_simd` (256-bit)	2.327	32.44	6.35×

Table 4: `./bin/conv simd` on 2048×2048, $K = 3$, seed 1234, P-core CPU 0.

3.4 Width and Size Sweep

We swept the image size $\in \{128, 256, 512, 1024, 2048, 4096\}$ ($K = 3$, seed 1234) for each SIMD register width, measuring the speedup of `conv_simd` over `conv_naive` at every point. The 256-bit code is the main one. The 512-bit version is skipped as none of our CPUs supported 512-bit SIMD registers :(

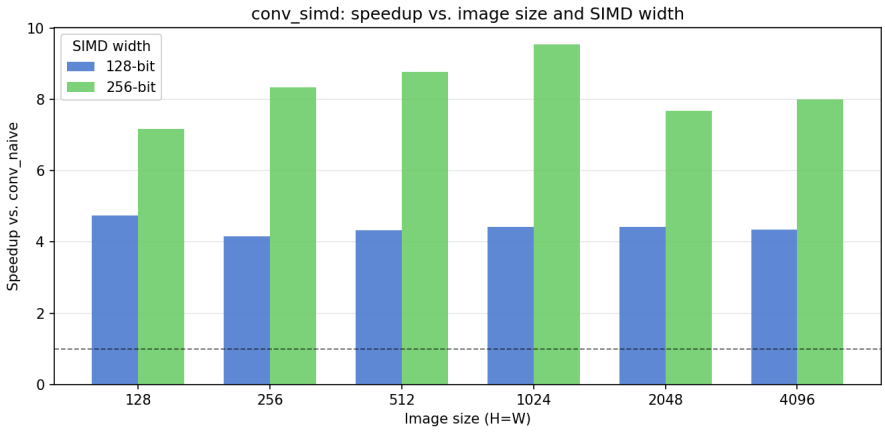

Figure 5: Speedup of `conv_simd` over `conv_naive` vs. matrix size, one line per SIMD width ($K = 3$).

3.5 Questions

1. **Instruction count change (naive → SIMD):** The naive implementation executes about 355.5M instructions per call, whereas the 256-bit SIMD implementation executes about 43.6M, roughly 8.2x fewer.
   The main reason is the vector width, one SIMD instruction processes 8 output columns at once instead of 1, so the inner loop runs 8x fewer times. On top of that, the `fmadd` instruction somewhat helps by merging the add and multiply instruction so that we achieve a bit more reduction in instruction size.

2. **Speedup:** We got a **6.35x speedup at 2048x2048, K=3**. The main contributors are the 256-bit vector operations and the `fmadd` instruction "fusing" the multiply and add operations. The speedup is a bit lower than the instruction reduction because the cpu still needs to load the same amount of data from memory regardless of single or vector loads. At K=3, there are only 9 multiply-adds per output pixel, so there isn't that much more computation relative to the data being loaded. When we increased the kernel size to 7, we saw bigger performance improvements (7.3x). Moreover, the naive implementation does sequential accesses which the hardware prefetcher handles well.

3. **Intrinsics used:** The following SIMD intrinsics were used:

- `_mm256_setzero_ps()` - initialises the register we use for calculating the accuracy to zero at the start of each column block. Cheaper than loading zeros from memory and the fact that it's vectorised.
- `_mm256_set1_ps(ker[ky*K + kx])` - broadcasts the scalar kernel weight into all 8 distinct 32-bit data parts of the 256-bit register. The weight is the same for every one of the 8 output columns being computed in parallel, so broadcasting once and reusing it in a vectorised manner is the best way to go about it.
- `_mm256_loadu_ps(ptr)` - loads 8 consecutive floats from the input. The `u` (unaligned) variant is used because the pointer includes an `ox+kx` offset that is not guaranteed to be 32-byte aligned. This is, again, a vector memory load which helps performance.
- `_mm256_fmadd_ps(w, pix, acc)` - fused multiply-add: computes `acc = w*pix + acc` in a single instruction instead of a separate multiply and add. We use this instead of separate multiply and add functions because this does it in one go instead of two different instructions and so is faster.
- `_mm256_storeu_ps(ptr, acc)` - writes all 8 results back to the output array. One store per 8 outputs instead of 8 separate scalar stores. Unaligned for the same reason as the load.

4 Task 1D: Combined Optimizations

4.1 Implementation

`conv_optimized` combines 256-bit SIMD with a 4-wide unroll of the SIMD loop, processing $4 \times 8 = 32$ output columns per outer iteration. Four independent `__m256` accumulators (`acc0`–`acc3`) each accumulate a separate 8-wide vector over the K^2 kernel iterations, and all four stores are issued back-to-back at the end. No output tiling is used.

Several combinations were evaluated before settling on this design:

- **Tiling only** - essentially no speedup ($\approx 1\times$ across all sizes).
- **SIMD only** - strong but decreases with image size (memory bound).
- **Tiling + SIMD** - slower than SIMD alone across all sizes.
- **Tiling + SIMD + Unrolling** - good, but still trails the no-tile version.
- **SIMD + Unrolling (chosen)** - consistently highest speedup at every size.

4.2 Results

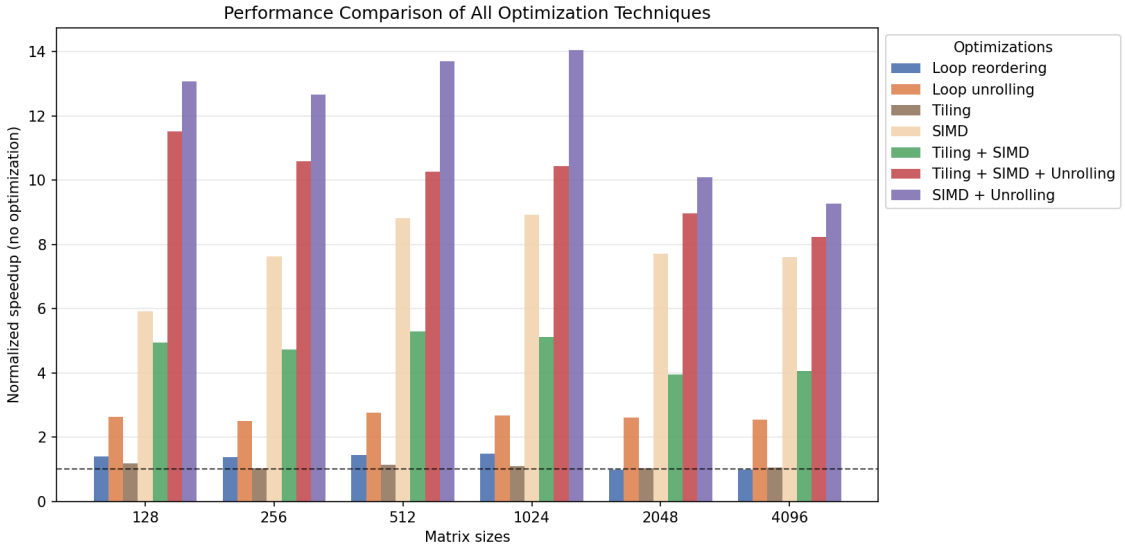

Figure 6: Speedup of all optimization stages over `conv_naive` vs. image size ($K = 3$, P-core CPU 0). `conv_optimized` is SIMD + Unrolling.

Stage	**Speedup @ 2048^2**	**Speedup @ 4096^2**
Loop reordering	$\approx 1.0\times$	$\approx 1.0\times$
Loop unrolling	$\approx 2.6\times$	$\approx 2.6\times$
Tiling ($T = 64$)	$\approx 1.0\times$	$\approx 1.0\times$
SIMD (256-bit)	$\approx 7.7\times$	$\approx 7.6\times$
SIMD + Unrolling	$\approx 10.2\times$	$\approx 9.3\times$

Table 5: Approximate speedup of each stage from the plot, $K = 3$.

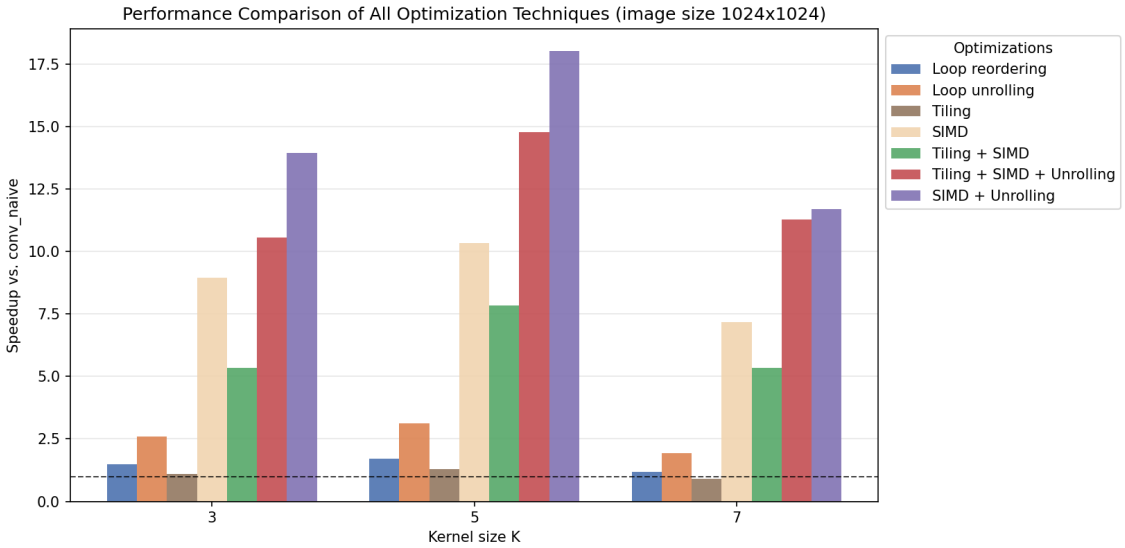

Figure 7: Speedup of all optimization stages over `conv_naive` vs. kernel size $K \in \{3, 5, 7\}$, at a fixed image size (1024^2).

Stage	K=3	K=5	K=7
Loop reordering	1.48x	1.72x	1.19x
Loop unrolling	2.60x	3.13x	1.92x
Tiling	1.10x	1.30x	0.90x
SIMD	8.96x	10.33x	7.16x
Tiling + SIMD	5.35x	7.83x	5.34x
Tiling + SIMD + Unroll	10.57x	14.79x	11.28x
`conv_optimized`	13.95x	18.01x	11.70x

Table 6: Speedup vs. `conv_naive` by kernel size, image size 1024^2 (mean of 10 runs).

Every stage peaks at $K = 5$, not $K = 3$ or $K = 7$: `conv_optimized` rises from 13.95× to 18.01×, then falls to 11.70× at K=7, below its own K=3 number; `tile` drops under 1× there. Our register blocking and unroll widths are sized for small K; the K^2 inner loop (9 iterations at K=3, 49 at K=7) outgrows that fixed unroll factor and starts paying for the overhead, while `conv_naive` has no such structure to outgrow and scales smoothly.

4.3 Discussion

Why tiling does not help. At $K = 3$, the naive loop already reads the input in a nearly linear fashion: the hardware prefetcher readily handles the short stride between successive 3×3 patches, so the L1-D miss rate is already very low (≈ 0.13 MPKI, as measured in Task 1B). Adding tiling disrupts this linear stream into short bursts with inter-tile jumps, introducing overhead without a corresponding cache benefit. The tiling loop nest also adds branch and index overhead. As a result, tiling alone gives no measurable speedup, and combining it with SIMD actually *reduces* performance relative to SIMD alone (Tiling + SIMD $\approx$ 5–6× vs. SIMD alone $\approx$ 7–11×).

Why SIMD + Unrolling wins. The 4-wide SIMD unroll (32 output columns per iteration) gives two independent benefits that compound:

- **Vector throughput**: each `fmadd` processes 8 floats at once, cutting instruction count by $\sim 8\times$.
- **Parallel instructions across vectors**: the four independent accumulators give the out-of-order engine four independent instructions to optimize, hiding some of the latency.

Together they deliver $\approx 10\times$ at 2048^2 and $\approx 9.3\times$ at 4096^2, with a decline at large sizes because the input data is still streamed sequentially and the prefetcher keeps the pipeline fed.

Why the speedup decreases with image size. At small sizes (128^2, 256^2) the entire input and output fit in L2/L3 cache, and the gain is highest ($\approx$ 13–14$\times$). As the image grows beyond L3 capacity ($\sim$24 MB at 4096^2), DRAM bandwidth becomes the limiting factor and the speedup settles to $\approx$ 9–10$\times$, since vector loads from DRAM also saturate the memory bus in a similar manner.